

NAIL

Not An Inform Library
Version 1.3
by Fredrik Ramsberg

What is NAIL?

It's simliar to a standard library for Inform 6, like a scaled-down version of PunyInform, meant for writing games for platforms with very little RAM. However, it lacks much of the finesse of PunyInform, and it's a less clean approach. Essentially, you get two source code files to start with (`game.inf` and `grammar.inf`), you edit them until you're happy, use two Ruby-scripts to transform the files to a compilable form, then compile them to a z3-file.

How is NAIL better than PunyInform?

- It's smaller. The minimum size of a compiled game is less than 11 KB, which is less than half the minimum size of PunyInform.

How is NAIL worse than PunyInform?

- The parser is a *lot* simpler. It doesn't support any parse tokens that use a routine (e.g. scope tokens or general parse routines), giving orders to NPCs, multiple objects like "get pear and apple" (Special case: "get/drop all [but x]" is supported, and secondary/indirect object is supported, e.g. "put x in y").
- Updating the library to a new version for an existing project is harder, because there is no clear separation between game code and library code. However, *most* of the library code is located in `nail.h` and `parser.h`, which you shouldn't need to touch, so if there's a new version of NAIL, which includes changes to these two files, you can copy the new versions of these files into your lib folder, overwriting the old versions.
- Unlike PunyInform, there is no test suite that ensures that new versions don't ruin existing functionality.
- The only supported output format is z3.
- The player can't be in a closed, non-transparent container or in/on a nested container/supporter.
- There is no support for properties `add_to_scope`, `react_before` or `react_after` (there are ways to accomplish the same effect, but with messier code, see chapter on limitations).
- Parser error messages, as well as action messages, are cruder and more generic.
- The documentation is laughably bad. Expect to dig in source code files to find stuff out.
- You need to have Ruby installed (used every time you compile a game).

Getting started

Copy the folder `examples/template`. Give the new folder a name you like. Edit the paths at the beginning of the file `build.bat` (or, if using Linux or MacOS: `build.sh`). Start a command prompt. Navigate to your new folder. Type `build.bat` or `build.sh`. If all goes well, you should see four new generated source code files appearing in your folder, and two new z3 files - one with `DEBUG` enabled and one without.

The files

The standard project consists of these files (found in `examples/template`):

- **game.inf** - The main source code of the game. Expect to edit this file a lot.
- **grammar.inf** - The action routines and the grammar for the game. Expect to edit this quite a bit.
- **abbreviations.h** - Abbreviations to improve text compression. Included by `game.inf`. From time to time, you'll want to use `Zabbrev` to figure out the best abbreviations for your game.
- **build.bat** - A batch file for Windows, to perform the steps needed to build Z-code files. This is what it does:
 1. Run `bin/build_dictionary.rb`, producing `generated_dictionary.inf`
 2. Run `bin/transform_code.rb`, producing `generated_game.inf`, `generated_grammar.inf`, and `generated_parser.h`
 3. Use `inform6.exe` to compile `generated_game.inf`, which in turn includes `nail.h`, `generated_dictionary.inf`, `generated_parser.h` and `generated_grammar.inf`, to produce `game_debug.z3` and `game.z3`
- **build.sh** - a shell script to do the same thing as `build.bat`

All projects need to have access to the `bin` folder. In this folder, you find these Ruby files:

- **build_dictionary.rb** - Reads the source code files that contain dictionary words (`game.inf`, `grammar.inf` and `parser.h`) and creates/overwrites a file called `generated_dictionary.inf`
- **transform_code.rb** - Reads `generated_dictionary.inf`, then reads `game.inf` and creates/overwrites `generated_game.inf` where each dictionary word has been replaced by the corresponding constant from `generated_dictionary.inf`, then it reads `grammar.inf` and creates/overwrites `generated_grammar.inf`, and reads `parser.h` and creates/overwrites `generated_parser.h` in the same manner.

All projects also need to have access to the `lib` folder. In this folder, you find these files:

- **nail.h** – Contains most of the library code.
- **parser.h** – Contains the parser code and some supporting routines.

Actions and Grammar

In the file `grammar.inf`, create a grammar table (e.g. `g_take`) for each verb (synonyms use the same grammar table though). Each line in the grammar table consists of an action, followed by up to three grammar tokens or dictionary words. Alternative prepositions are not supported. E.g. supporting "PUT X IN/INTO Y" requires two grammar lines.

The available tokens are:

- **T_NOUN:** Matches an object, which is then put into the `noun` variable.
- **T_SECOND:** Matches an object, which is then put into the `second` variable.
- **T_DIRECTION:** Matches a direction, which is then put into the `direction` variable. (Directions are not objects, in any way).
- **T_TOPIC:** Matches any number of words. Must be followed by a preposition, or be the last token on the line.
- **T_ALL:** Matches 'all' or 'everything'
- **T_BUT:** Matches 'but' or 'except'
- **T_SET_META:** Has to be the first entry in a grammar table. Doesn't match anything, but sets `meta = true` for the entire grammar table.

Dictionary

There is no dictionary. Instead each word is represented by a 16-bit checksum of that word - a number which is either in the range [-2000, -977] or [1024, 32767] (a range of numbers around 0 is deliberately avoided, because it's convenient to be able to tell dictionary word checksums from NULL, false, true, object numbers, verb numbers, and some other values).

When the player has typed a command, the parser calculates and stores the checksum for each typed word in the parse table. The game author can compare these values to dictionary words in the code, but the build process will in fact replace the dictionary words with constants representing the checksum of these words, when generating `generated_game.inf`, `generated_grammar.inf`, and `generated_parser.h`.

The names of the constants for different words are derived from the words, e.g. 'east' gets the constant name `W_EAST`. A single quote character is transformed to `_Q_`, so 'tom^s' gets the constant name `W_TOM_Q_S`.

If you have a word which can't be turned into a constant in this automated manner, maybe because it contains illegal characters or because the name is already in use, you can decide on a constant name yourself, and declare it in a comment in `game.inf`. This is what it may look like:

```
! 'to' = W_TO_WORD
```

This particular line avoids having the constant for 'w//' collide with the `w_to` property. Note that you can only declare one such constant per line.

If a certain constant name would contain special characters, and you have not assigned a special name to it in this way, it will be called `W_SPECIAL_WORD_n`, where `n` is 0, 1, 2 etc.

Note: Encoding the words as checksums involves finding an encoding factor that give all the words distinct checksums. As the list of words your game recognizes grows, it is possible that the `build_dictionary.rb` script will fail to find a factor which makes all words get distinct values. If this happens, you will see that `build_dictionary` reports some collisions. The exact collisions are listed in `generated_dictionary.inf`. If this happens, read the list of collisions, and decide if it's important that the parser can tell these words apart. E.g. if there is a collision between two words that are only used for two different static or scenery objects, which will never be in scope at the same time, it doesn't matter. If you have collisions that matter, consider leaving out a synonym for an object or a verb, or even renaming an object. Note that a collision between a verb and a noun is typically okay – the parser can't confuse them.

Limitations of NAIL, compared to PunyInform

- `z3` is the only target supported
- Parser limitations:
 - NOT SUPPORTED: Multiple objects or plural, except for "take all [but x]" and "drop all [but x]". E.g. "take pear and apple" isn't supported. Secondary object is supported, e.g. "put apple in box".
 - NOT SUPPORTED: Giving NPCs orders, e.g. "John, pull lever".
 - NOT SUPPORTED: grammar tokens involving routines (scope tokens, general parse routines etc)
 - NOT SUPPORTED: grammar lines with alternative prepositions
- The player must never be in a closed, non-transparent container. Workaround: Move the player to a room that represents the inside of the container.
- Nested enterable objects are not supported (e.g. a seat in a car).
- Regular grammar tables are not in use, and you can't use `Verb` or `Extend`. You have to alter the grammar in the file where it is defined (typically `grammar.inf`).
- Since there is no dictionary, you can't print dictionary words, or check for verb flag etc.
- Scope can only be calculated for the player, not for other actors.
- Attributes `edible`, `talkable` are not supported.
- Properties `orders`, `grammar`, `life`, `capacity`, `inside_description`, `list_together` are not supported.
- Property `add_to_scope` is not supported. Use entry point `InScope` instead.
- Properties `react_before` and `react_after` are not supported. Use `before/after` in location, or `GamePreRoutine/GamePostRoutine` instead.
- Properties `when_on`, `when_off`, `when_open`, `when_closed` are not supported. Use `initial` and `describe` instead.

Differences to PunyInform

- Directions are not considered objects, in any way.
- If a command involves a direction, it is stored in the `direction` variable.
- `noun/second` are not used to store anything but object numbers.
- The first word of a topic (not counting articles) is stored in `first_topic_word`.
- There is no grammar token for a number, but you can use a `T_TOPIC` token instead, and use `x = TryNumber(consult_from)` to extract a number from the first word, or decide that it's not a number (-1000 means not a number, 0-10000 means a number).
- Simple doors (an optional feature in PunyInform) is always available in NAIL. There is no constant to enable or disable it.
- Since there is no `life` property, you use `before` to react instead (`##Ask`, `##Tell`, `##Give`, `##Show`, `##Attack`, `##ThrownAt`, `##Answer`).
- `##Give` and `##Show` have the actor as noun, so an actor's `before` routine can react to these actions. (Just like in PunyInform / Inform 6, `##Ask` and `##Tell` have the actor as noun).
- Instead of using the `name` property, you use the `nouns` property to store a list of synonyms for an object. If you need more than four synonyms, use the `longnouns` property instead - an individual property, which takes up more space and is slower to access, but can hold up to 32 values.

- Property `invent` is only called once per object, not with different `invent_stage` values. It can either print all information about the object and return true, or don't print anything and return false, or (sometimes useful) print something (e.g. "full " or "empty ") and return false to have the object printed in the normal way.
- The author must always set `scope_modified = true` when doing something which may affect what's in scope. (This is optional in PunyInform, see `OPTIONAL_MANUAL_SCOPE`. Here, it's mandatory.)
- As an author, you must give the `reactive` attribute to all objects that provide the `each_turn` property. (This is optional in PunyInform, see `OPTIONAL_MANUAL_REACTIVE`. Here, it's mandatory.) If you define `OPTIONAL_REACTIVE_PARSE_NAME`, you must also give `reactive` to all objects that provide `parse_name`.
- Lookmode commands use the Look verb: "look short/superbrief", "look normal/brief", "look long/verbose". Also, the ability to control lookmode is optional – set `OPTIONAL_LOOKMODE` to enable it.
- Disambiguation can only be done by having the user select one of the objects from a numbered list, using a special prompt. Failing to select an object means the first object in the list is selected.

Supported user-defined constants:

- Story
- Headline
- DEBUG
- `INITIAL_LOCATION_VALUE`
- `GOTOSUB_BUFFER_SIZE` - In DEBUG mode, set to length of longest object name
- `MAX_FLOATING_OBJECTS`
- `MAX_SCOPE`
- `MAX_WHICH_OBJECTS`
- `NO_PLACES` - See DM4
- `OPTIONAL_DARKNESS` (Opposite of PunyInform's `OPTIONAL_NO_DARKNESS`)
- `OPTIONAL_LOOKMODE` - Support "look short/normal/long" to choose lookmode
- `OPTIONAL_PRONOUNS` - Support it/him/her/them in commands
- `OPTIONAL_REACTIVE_PARSE_NAME` - See PunyInform
- `OPTIONAL_SCRIPT` - Include support for "script on/off" and "recording on/off" and "replay"
- `RUNTIME_ERRORS` - See PunyInform
- `STATUSLINE_SCORE` - See PunyInform
- `STATUSLINE_TIME` - See PunyInform
- Scoring:
 - `NO_SCORE` - See PunyInform
 - `MAX_SCORE` - See DM4
 - `OBJECT_SCORE` - See DM4
 - `ROOM_SCORE` - See DM4
 - `OPTIONAL_SCORED` - See PunyInform
 - `OPTIONAL_FULL_SCORE` (`NO_SCORE` can't be defined) - See PunyInform
 - `TASKS_PROVIDED` (requires `OPTIONAL_FULL_SCORE`) - See DM4
 - `NUMBER_TASKS` (requires `TASKS_PROVIDED`) - See DM4

Supported entry points

(See DM4 for most of them)

- Amusing
- AfterLife
- DarkToDark
- DeathMessage
- DebugParseNameObject - See PunyInform
- GamePostRoutine
- GamePreRoutine
- InScope
- IsARoom - See PunyInform
- LookRoutine
- NewRoom
- PrintTaskName (requires OPTIONAL_FULL_SCORE)
- TimePasses

Using PunyInform Extensions

- **Cheap Scenery** - works. To make the 'ctest' debug verb work, you have to add grammar to your game to make "ctest" trigger the ##DebugCheapScenery action.
- **Flags** - works
- **Menu** - works
- **Quote Box** - works
- **Talk Menu** - works. You have to add grammar to your game to make "talk to noun" and "talk noun" trigger the ##Talk action. In Debug mode, you have to add grammar to make "tmtest" and "tmtest noun" trigger the ##TMTest action.
- **Wait Time** - Doesn't work. One would need to create a new reworked version which doesn't use a general parse routine.